Adaptive Reverse Proxy with Latency-Aware Routing

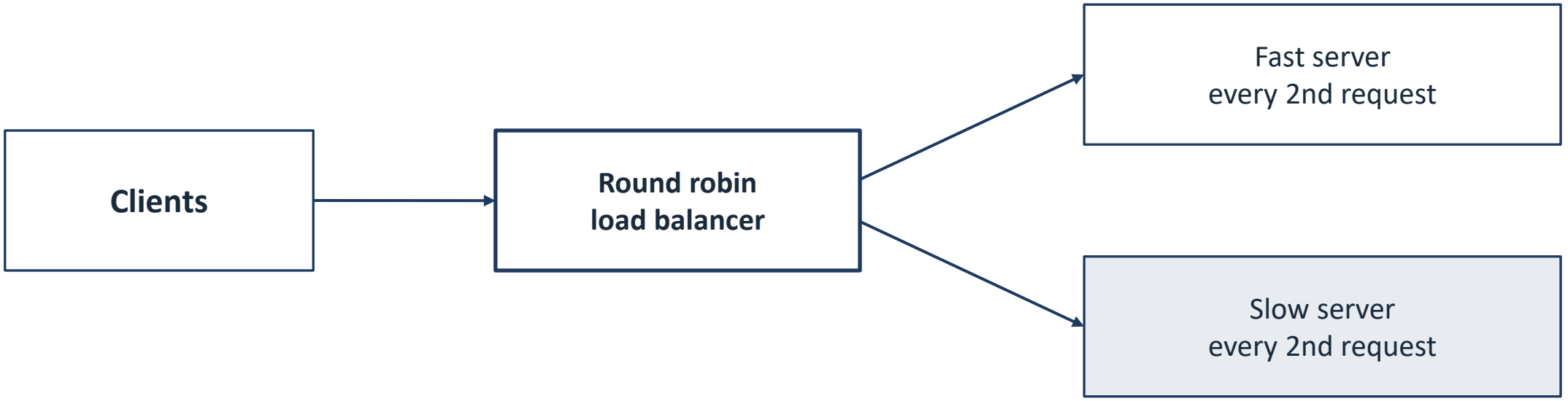
Computer Networks project demonstration

Presented by: Seshrut Jain, Vamsi Indrakanti, Joka srinivasa sai krishna sridda

Roll numbers: 24BAI0117, 24BCE0924, 24BCE0897

Problem: static load balancing ignores server speed

Round robin gives every server the same share, even when one server is slow.



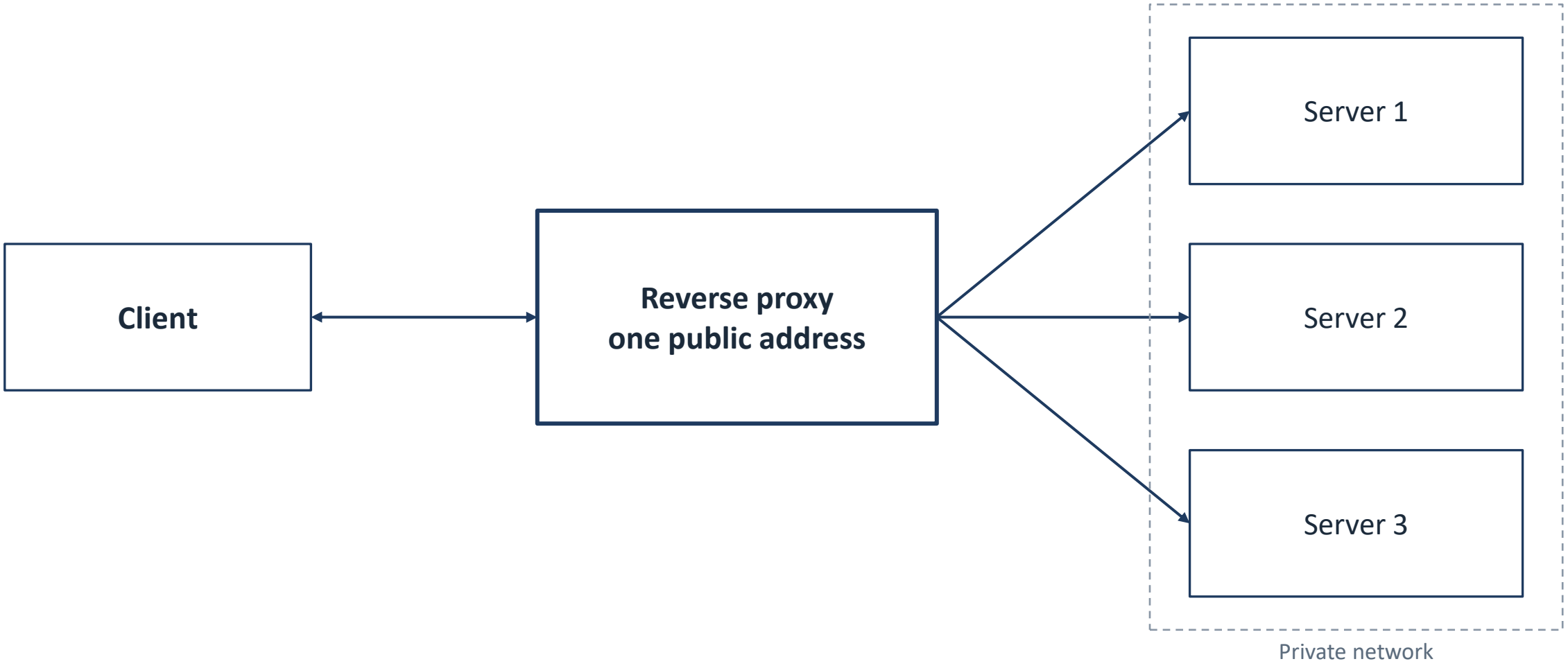
With two equally selected servers, about half of the requests may go to the slow one.

Objective

Choose backends using recent latency and current load.

- Measure the response time of every forwarded request
- Favour servers with lower recent latency and lower load
- Keep working when a server fails
- Show it all live on a dashboard

Background: what is a reverse proxy



Clients talk only to the proxy. The servers stay hidden behind it.

Background: forward proxy vs reverse proxy

	Forward proxy	Reverse proxy
Sits in front of	Clients	Servers
Hides	The client	The servers
Set up by	Client side or organisation	Server owner
Typical use	Filtering, access control	Load balancing, caching

Our project builds a reverse proxy.

Background: common load balancing methods

Round robin

Each server takes a turn, in a fixed order

Least connections

Server with the fewest open requests

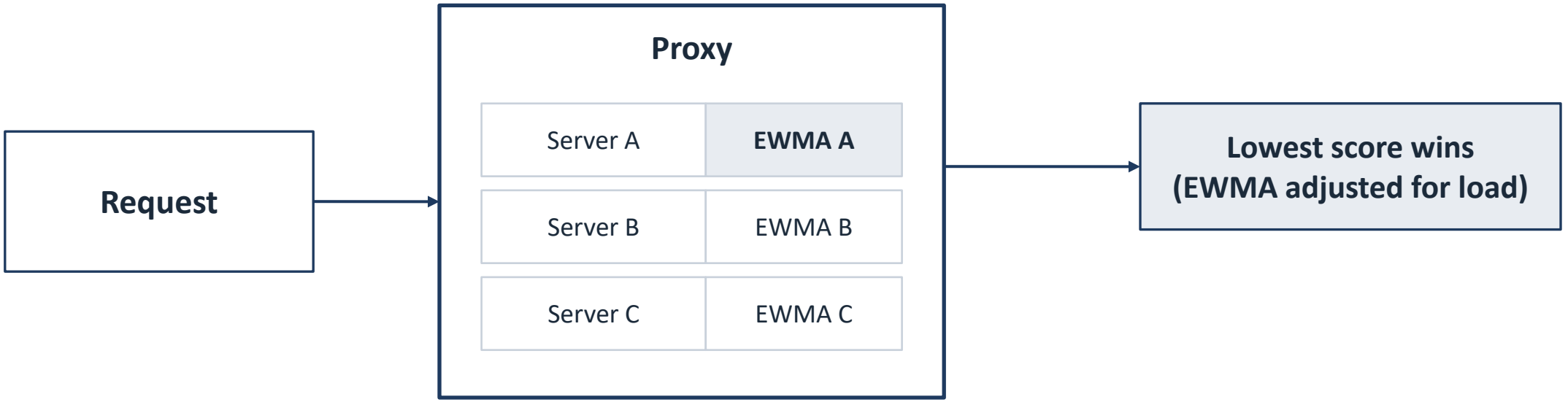
IP hash

Same client IP always goes to the same server

These standard policies do not directly choose a backend using measured response latency.

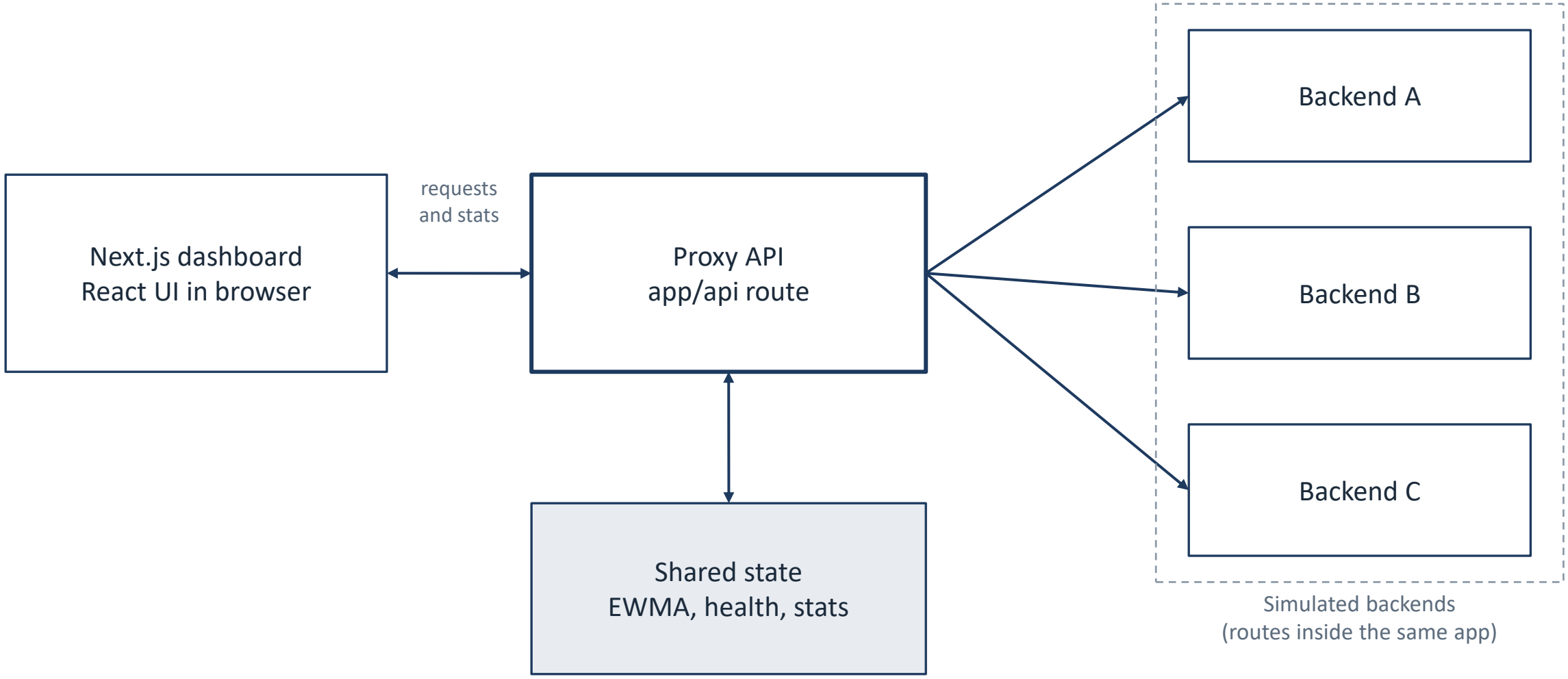
Our idea: latency-aware routing

Send each request to the server with the best recent latency score.



Estimates are updated after every response. Full rule on slide 12.

System architecture



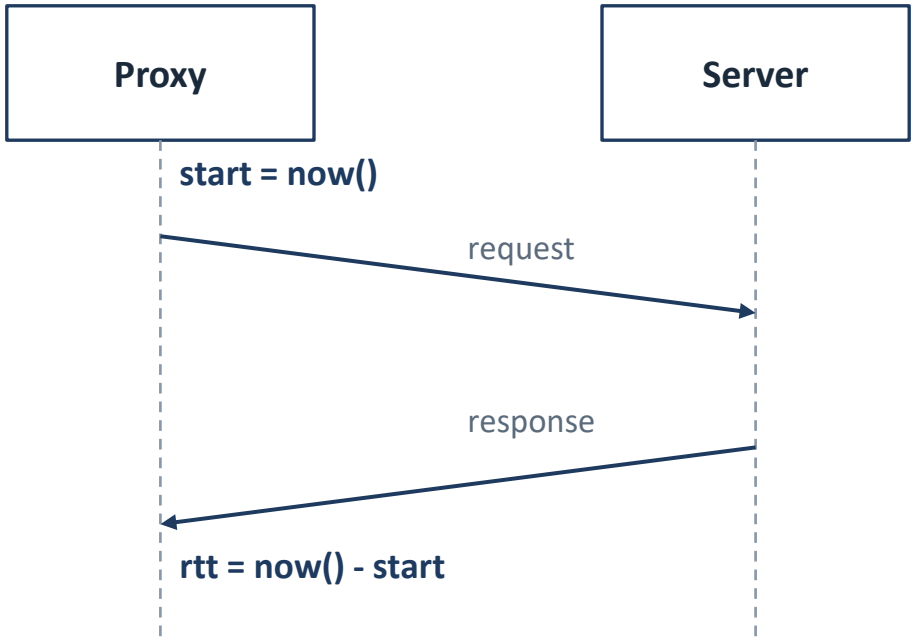
Request flow



Steps 2 to 5 repeat for every request.

Measuring latency

Time each forwarded request at the proxy.



Recent requests

Newest first. The last 30 requests are kept.

Time	Routed to	Response time	Strategy	Failover
9:55:53 AM	Server C	286 ms	lowest-latency	No
9:55:53 AM	Server A	115 ms	lowest-latency	No
9:55:53 AM	Server A	120 ms	lowest-latency	No
9:55:53 AM	Server B	191 ms	lowest-latency	No
9:55:53 AM	Server A	159 ms	lowest-latency	No
9:55:53 AM	Server A	111 ms	lowest-latency	No
9:55:52 AM	Server A	100 ms	lowest-latency	No
9:55:52 AM	Server A	126 ms	lowest-latency	No

Recent requests table, adaptive run E1-AD-2 (real dashboard capture)

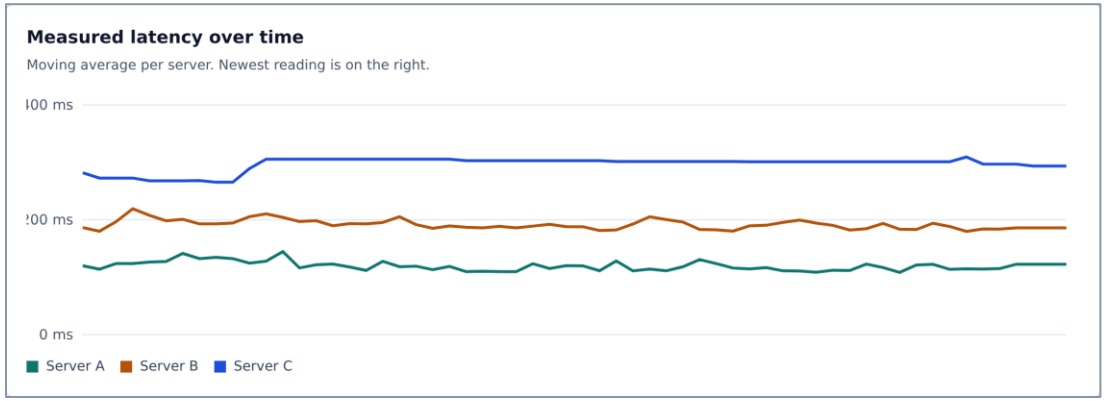
Measures the proxy-to-backend request/response interval (the timer wraps the fetch call to the backend), not total client latency. In this demo it includes the simulated delay (base ± 10% jitter + 5 ms per request in flight) and the local HTTP hop.

Smoothing with EWMA

$$new = \alpha \times sample + (1 - \alpha) \times old$$

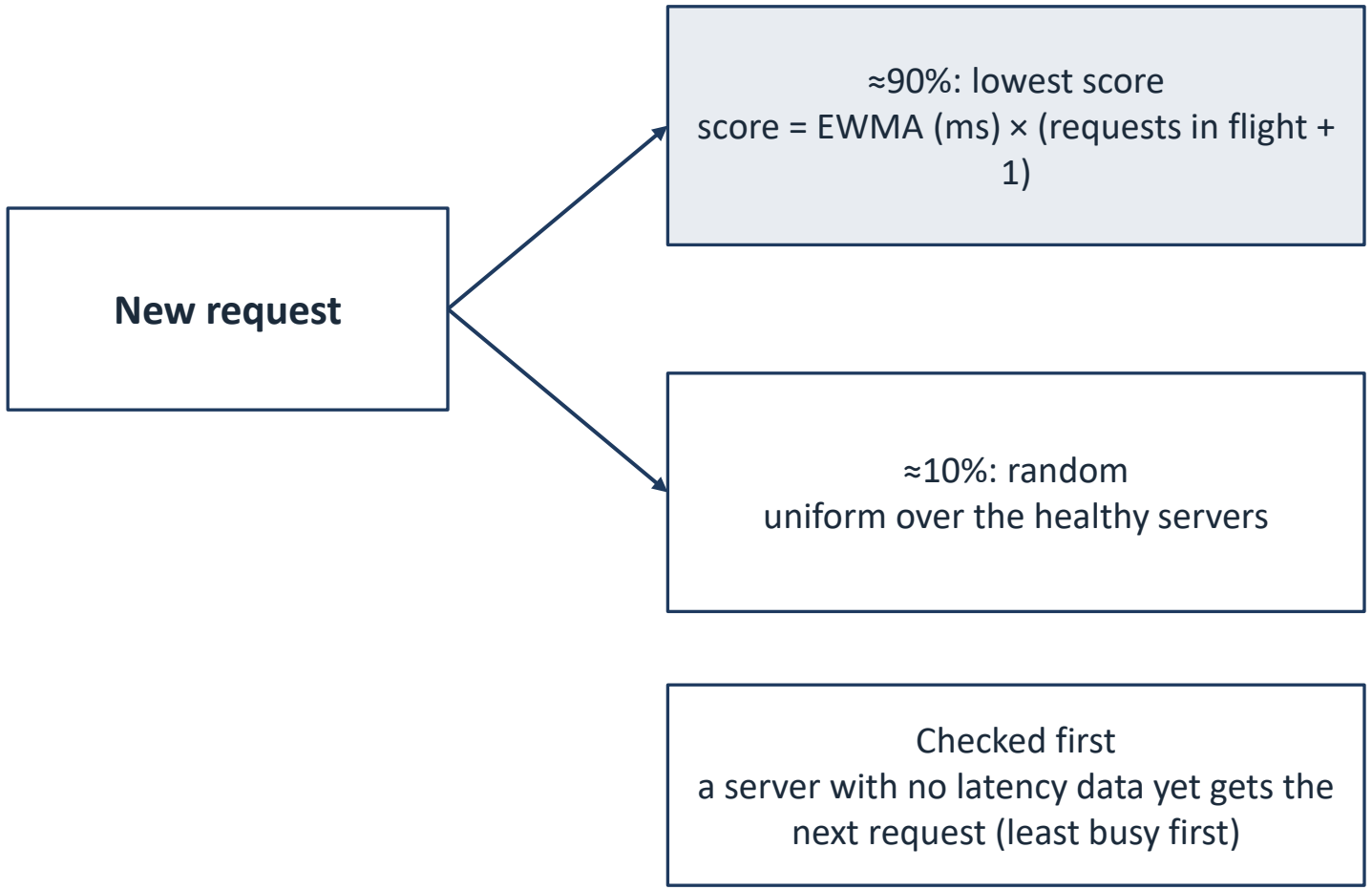
α = 0.3 (the first reply sets the starting value)

- One slow reply moves the average only 30% of the way: spikes are damped, not removed
- A real slowdown shows within a few requests
- Needs only one stored number per server



Moving average per server, adaptive run E1-AD-2 (real dashboard capture)

Selection strategy



Why explore?

- A slow server that recovers gets traffic again
- A recovered server starts with no estimate, so it is tried first
- Busy servers score higher, so load spreads (score is ms × requests, not plain ms)

Failover and health checks

Server returns an error or times out (2 s)

↓

Proxy retries on the other healthy servers, one by one

↓

3 failures in a row: removed from pool

↓

Probed at most every 3 s (when traffic arrives) until it answers



Servers

Add or remove servers, change how slow they are, or take one down to test failover.

Add server

Server	Status	Simulated latency	Measured latency (average)	In flight	Requests served	Failures	Control
Server A	Down	80 ms	131 ms	0	51 (27%)	4	Bring up · Remove
Server B	Healthy	150 ms	197 ms	0	97 (51%)	0	Take down · Remove
Server C	Healthy	250 ms	295 ms	1	41 (22%)	0	Take down · Remove

Servers

Add or remove servers, change how slow they are, or take one down to test failover.

Add server

Server	Status	Simulated latency	Measured latency (average)	In flight	Requests served	Failures	Control
Server A	Healthy	80 ms	114 ms	1	132 (39%)	4	Take down · Remove
Server B	Healthy	150 ms	174 ms	0	154 (45%)	0	Take down · Remove
Server C	Healthy	250 ms	279 ms	0	54 (16%)	0	Take down · Remove

Real captures, run E3-r2. Top: Server A after Take down (Down, 4 failed attempts). Bottom: after Bring up (Healthy again).

Measured in 2 runs at 10 pkt/s: marked down 0.7–1.2 s after Take down; back in the pool 0.4–1.3 s after Bring up; 0 client-visible failures (4 failed attempts, all retried). Take down gives an instant 503; the 2 s timeout path was not tested. If no healthy server is left, the request fails.

Tech stack

Layer	Technology	Purpose
Framework	Next.js 14.2.15	Dashboard pages and proxy API routes
User interface	React 18.3.1	Live traffic view, charts, controls
Language	TypeScript 5.5.4	Typed proxy, strategy and server code
Styling	Tailwind CSS 3.4.13	Layout and component styling
Animation	framer-motion 11.11.1	Packet animation in the topology diagram

Code structure

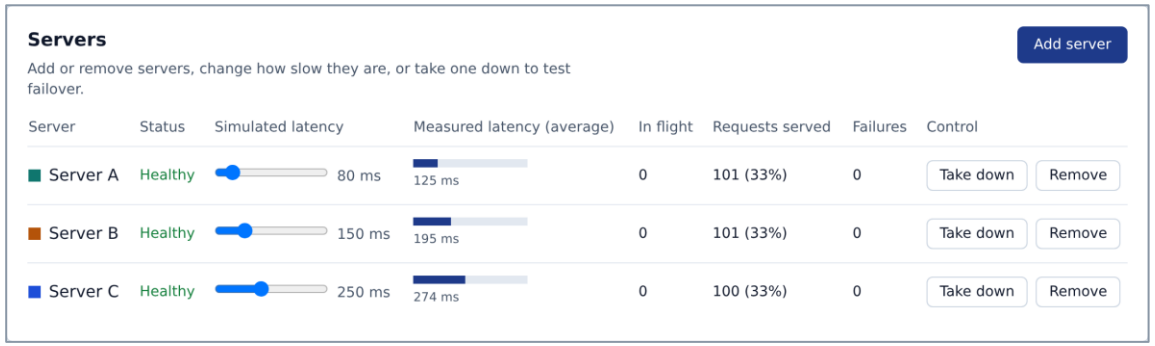
app/api/	components/ + hooks/	lib/	lib/strategies/
proxy, batch, backend, admin, stats and strategy routes	dashboard UI, topology diagram, polling and traffic simulator	proxy.ts, backend-client.ts, backend-stats.ts, health-check.ts, registry.ts, config.ts	round-robin.ts and lowest-latency.ts

```
$ tree app/api lib      # generated from the project folder (terminal output)

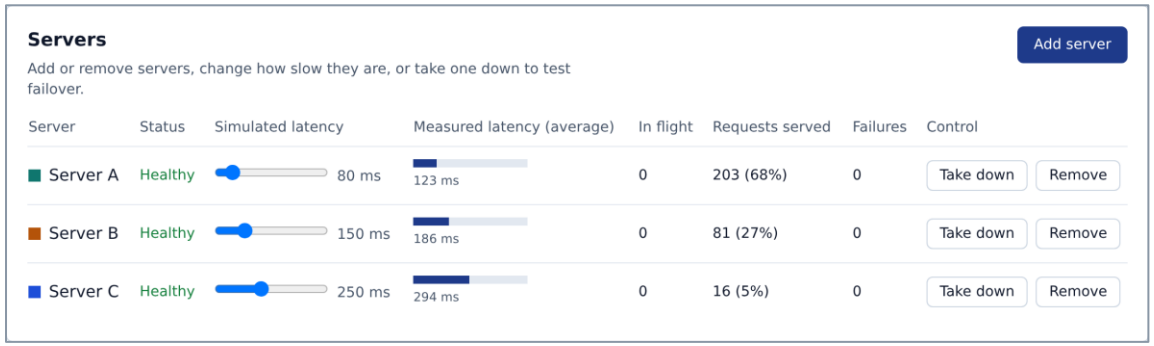
app/api/  route.ts files                          lib/
  admin/backends  (12)                            ├─ strategies/
  admin/backends/[id]  (32)                       │  ├─ index.ts  (9)
  backends/[id]  (30)                             │  ├─ lowest-latency.ts  (34)
  proxy  (17)                                     │  ├─ round-robin.ts  (9)
  proxy/batch  (22)                               │  └─ types.ts  (4)
  stats  (19)                                     ├─ api-client.ts  (53)
  strategy  (16)                                  ├─ backend-client.ts  (26)
                                                  ├─ backend-stats.ts  (29)
components/  16 dashboard files                   ├─ batch.ts  (22)
  (incl. topology/ diagram)                       ├─ chart-path.ts  (20)
hooks/  3 files                                   ├─ config.ts  (45)
                                                  ├─ health-check.ts  (23)
(n) = lines of code                               ├─ packet-timing.ts  (11)
                                                  ├─ proxy.ts  (41)
                                                  ├─ rate.ts  (9)
                                                  ├─ registry.ts  (109)
                                                  ├─ simulated-delay.ts  (11)
                                                  └─ types.ts  (50)
```

Generated from the real project folder (terminal output). Each routing strategy is its own file under lib/strategies/.

Experiment: round robin vs adaptive



Round robin, run E1-RR-2: sent 302, served 302, failed 0. Served A 101, B 101, C 100.

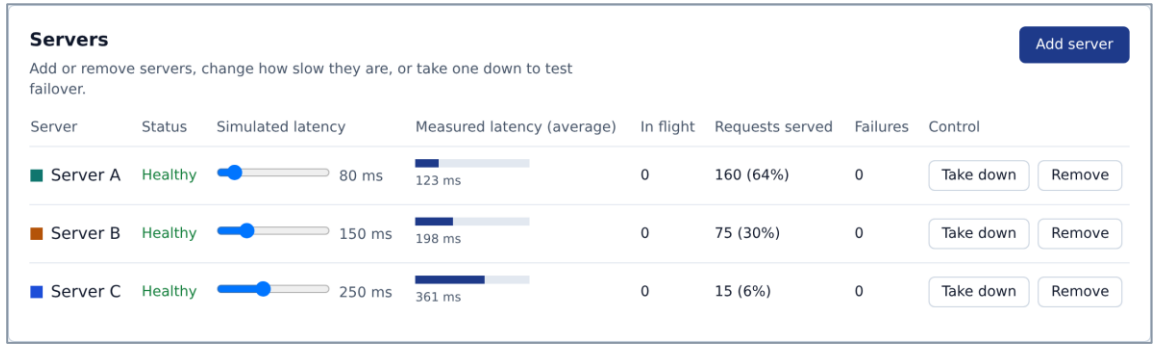


Adaptive, run E1-AD-2: sent 300, served 300, failed 0. Served A 203, B 81, C 16.

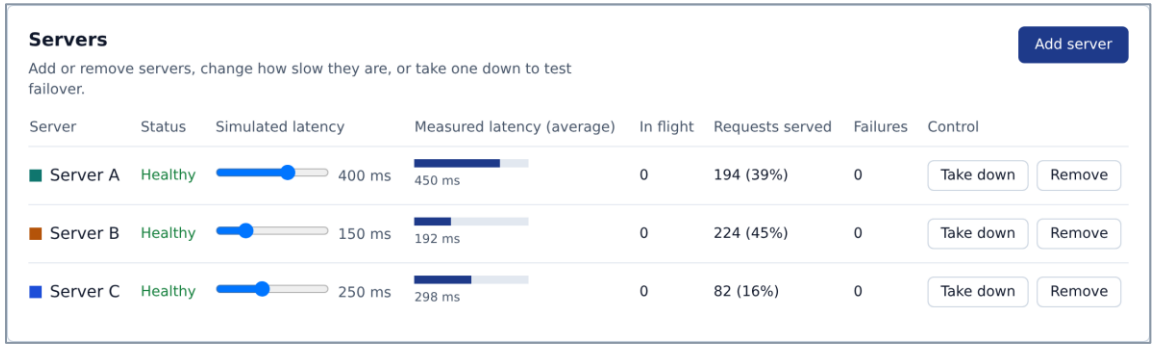
Same setup: 10 pkt/s configured, 30 s, delays A 80 / B 150 / C 250 ms	Round robin	Adaptive
Requests sent / served / failed, runs 1–3	302/302/0 · 302/302/0 · 300/300/0	303/303/0 · 300/300/0 · 300/300/0
Share to Server A (fastest), runs 1–3	33% · 33% · 33%	59% · 68% · 67%
Share to Server C (slowest), runs 1–3	33% · 33% · 33%	8% · 5% · 4%
Mean proxy-measured latency, runs 1–3	246 · 211 · 212 ms	191 · 149 · 150 ms
p95 proxy-measured latency, runs 1–3	393 · 327 · 323 ms	347 · 255 · 239 ms

Observed: adaptive sent 59–68% of requests to the fastest server and had lower mean proxy-measured latency in all 3 runs (22–29% lower). Run 1 of each strategy came first after server start and is slower (dev-mode warm-up). Latency is the proxy→backend interval; p95 is computed from every request in the run. Three runs on one machine with simulated servers: a limited demonstration, not statistical proof or a throughput benchmark.

Dashboard: before and after slowing a server



Before: Server A at 80 ms (adaptive, 250 requests since reset)



After: Server A slider set to 400 ms (500 requests since reset)

Experiment E2 (adaptive, 10 pkt/s)	Run r1 before	Run r1 after	Run r2 (before → after)
Server A configured delay (slider)	80 ms	400 ms	80 → 400 ms
EWMA A / B / C at capture (ms)	123 / 198 / 361	450 / 192 / 298	A: 116 → 428
Cumulative share A / B / C (%)	64 / 30 / 6	39 / 45 / 16	66 / 29 / 5 → 39 / 44 / 17
Share of the 250 requests sent after the change (%)	–	14 / 60 / 27	13 / 58 / 29
Mean proxy-measured latency	173 ms	258 ms	153 → 258 ms

Observed: after Server A was slowed to 400 ms its EWMA rose to about 430–450 ms and it received only 13–14% of the next 250 requests, down from 64–66% in the first 250. Server B, now the lowest, took about 60%. A still gets some traffic through 10% random exploration and load spreading.

Each run: reset, 25 s at 10 pkt/s, change slider, 25 s more, no reset in between. r1 and r2 are separate sessions. Cumulative shares include pre-change requests, so they lag; the “after the change” row isolates new traffic. Mean latency covers each phase’s requests only.

Limitations and future work

Limitations

- State is in memory (lost on restart); one proxy instance only
- Backends are simulated inside the same app, no real upstream URLs
- The in-flight count is kept by the simulated backend, and load is counted in both the latency and the score
- Probes only run when traffic arrives; the 2 s timeout path was not tested
- No built-in p95 or throughput metrics; our results are 3 runs on one machine

Future work (proposed, not implemented)

- Shared state (e.g. Redis) so several proxy instances can coordinate; persistence depends on its configuration
- Forward to real upstream URLs, counting in-flight requests in the proxy
- Built-in p95 and throughput benchmark, more runs
- Weighted routing, TLS termination, timer-based health probes

Conclusion

- Implemented: a proxy that routes by measured latency (EWMA, $\alpha = 0.3$) and load, with 10% exploration, retries, health probes and a live dashboard
- Measured (3 runs, simulated servers): adaptive sent 59–68% of requests to the fastest server and had lower mean proxy-measured latency than round robin in every pair (e.g. 150 vs 212 ms)
- After slowing Server A to 400 ms, it received 13–14% of the next 250 requests
- Taking a server down caused no client-visible failures in 2 runs; it was out and back within about a second or two
- This is a limited demonstration, not a production benchmark or proof for all workloads

Thank you

Questions?
Live demo: npm install, npm run dev, then open http://localhost:3000